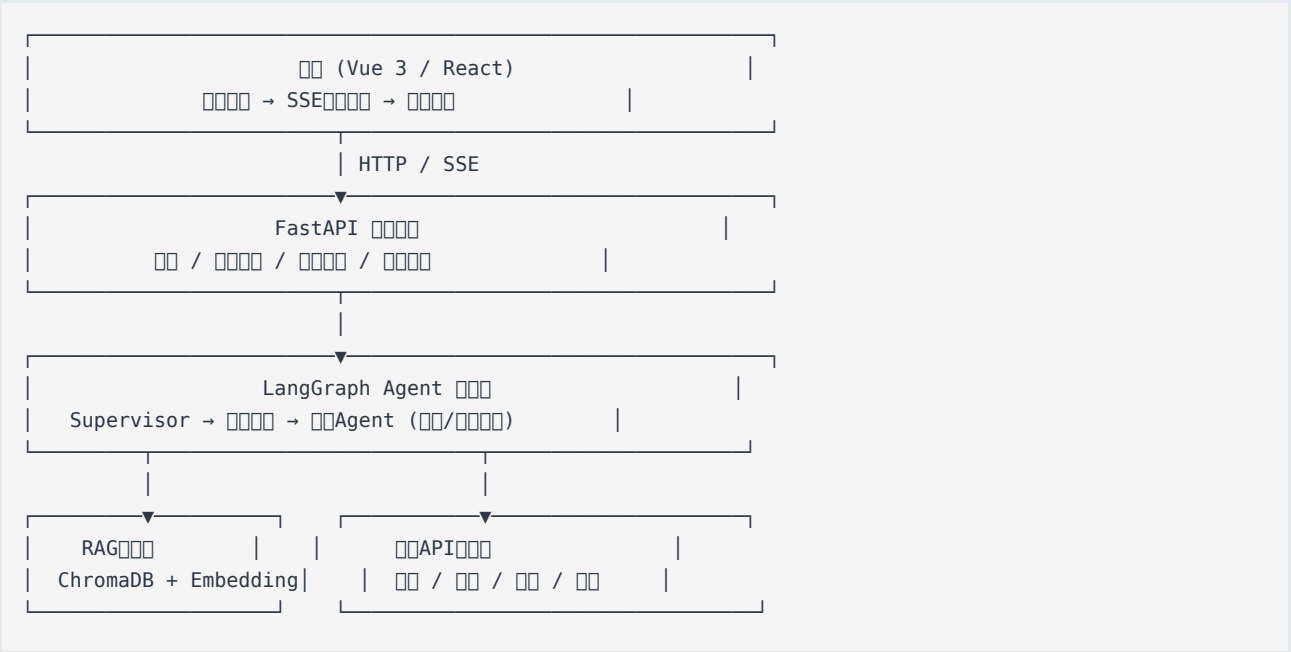
智能旅行规划系统

技术架构与实现文档

基于 LangGraph + FastAPI + RAG 的多 Agent 协作方案

一、总体架构概览

系统采用 前端 → 后端API → Agent编排层 → 工具/检索层 的四层架构：



二、技术栈选型

层次	技术选型	说明
前端	Vue 3 + TypeScript + Element Plus + Leaflet	或 React + Tailwind CSS
后端	FastAPI + Python 3.12	高性能异步API框架
Agent编排	LangGraph StateGraph + 动态Supervisor	多Agent协作核心
大语言模型	DeepSeek / Gemini / OpenAI	按成本和效果选择
向量数据库	ChromaDB (PersistentClient)	本地持久化，无需额外服务
Embedding模型	sentence-transformers (bge-small-zh-v1.5)	中文语义匹配
关系数据库	PostgreSQL / SQLite	用户、行程、审计日志
缓存	Redis	会话缓存、API响应缓存
部署	Docker Compose + Nginx	一键容器化部署

三、项目目录结构

参考多个成熟项目的组织方式：

```
travel-planner-agent/
├── backend/
│   ├── app/
│   │   ├── api/                # REST API + SSE □□
│   │   │   ├── auth.py         # JWT□□/□□
│   │   │   ├── trips.py        # □□□□□□
│   │   │   └── stream.py       # SSE□□□□
│   │   ├── db/                 # SQLAlchemy ORM□□
│   │   │   ├── models.py       # User, Trip, Preference
│   │   │   └── session.py      # □□□□□
│   │   ├── graph/              # LangGraph Agent□□
│   │   │   ├── state.py        # Agent□□□□
│   │   │   ├── nodes.py        # □Agent□□□□
│   │   │   ├── supervisor.py   # □□□□□
│   │   │   └── builder.py      # □□□□□□
│   │   ├── tools/              # □□□□
│   │   │   ├── weather.py      # □□API
│   │   │   ├── maps.py         # □□/POI
│   │   │   ├── flights.py      # □□□□
│   │   │   └── hotels.py       # □□□□
│   │   ├── rag/                # RAG□□
│   │   │   ├── embedder.py     # Embedding□□□□
│   │   │   ├── chroma_client.py # ChromaDB□□
│   │   │   ├── retriever.py    # □□□□
│   │   │   └── build_kb.py     # □□□□□□□
│   │   ├── services/           # □□□□□
│   │   │   ├── auth.py         # □□□□
│   │   │   ├── profile.py      # □□□□
│   │   │   └── itinerary.py    # □□□□
│   │   ├── schemas/            # Pydantic□□□□
│   │   └── config.py           # □□□□
│   ├── scripts/                # □□□□
│   │   ├── seed_all.py         # □□□□□□
│   │   └── process_docs.py     # □□□□□□
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── api/                # API□□□
│   │   ├── components/         # Vue/React□□
│   │   ├── stores/             # □□□□ (Pinia/Redux)
│   │   └── views/              # □□
│   └── package.json
├── data/
│   └── chromadb/               # ChromaDB□□□□□[reference:22]
├── docker-compose.yml
├── nginx.conf
└── .env.example
```

四、核心数据模型

4.1 数据库模型 (SQLAlchemy)

python

```
# backend/app/db/models.py

class User(Base):
    __tablename__ = "users"
    id = Column(UUID, primary_key=True, default=uuid4)
    email = Column(String, unique=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    preferences = Column(JSON)  # □□□□□{budget, pace, interests, cuisine}

class Trip(Base):
    __tablename__ = "trips"
    id = Column(UUID, primary_key=True, default=uuid4)
    user_id = Column(UUID, ForeignKey("users.id"))
    destination = Column(String)
    start_date = Column(Date)
    end_date = Column(Date)
    budget = Column(Float)
    travelers = Column(Integer)
    itinerary = Column(JSON)  # □□□□JSON
    status = Column(String)   # draft / approved / finalized
    created_at = Column(DateTime, default=datetime.utcnow)
    version = Column(Integer, default=1)  # □□□□

class AuditLog(Base):
    __tablename__ = "audit_logs"
    id = Column(UUID, primary_key=True)
    user_id = Column(UUID, ForeignKey("users.id"))
    action = Column(String)   # login / chat / export
    details = Column(JSON)
    timestamp = Column(DateTime, default=datetime.utcnow)
```

4.2 Agent状态模型 (LangGraph)

python

```
# backend/app/graph/state.py
from typing import TypedDict, List, Optional
from datetime import date

class TravelPlanRequest(TypedDict):
    destination: str
    start_date: date
    end_date: date
    budget: float
    travelers: int
    preferences: dict  # {pace: "relaxed", interests: ["culture", "food"]}

class AgentState(TypedDict):
    """LangGraph□□□□□□□"""
    request: TravelPlanRequest
    messages: List[dict]           # □□□□
    current_agent: str             # □□□□□Agent
    attractions: List[dict]        # □□□□□□□
    hotels: List[dict]             # □□□□□□□
    restaurants: List[dict]        # □□□□□□□
    transport: dict                # □□□□
    itinerary: dict                # □□□□
    validation_errors: List[str]   # □□□□
    approved: bool                 # □□□□□□□
```

五、RAG知识库实现

5.1 离线知识库构建流程

python

```python
# backend/app/rag/build_kb.py

from sentence_transformers import SentenceTransformer
import chromadb
from chromadb.config import Settings
from typing import List
import os

class KnowledgeBaseBuilder:
    def __init__(self, persist_dir: str = "data/chromadb"):
        # □□PersistentClient□□□□□□□[reference:27]
        self.client = chromadb.PersistentClient(
            path=persist_dir,
            settings=Settings(anonymized_telemetry=False)
        )
        self.collection_name = "travel_knowledge"
        self.collection = self.client.get_or_create_collection(
            name=self.collection_name,
            metadata={"hnsw:space": "cosine"}  # □□□□□[reference:28]
        )
        self.embedder = SentenceTransformer('BAAI/bge-small-zh-v1.5')  # □□□□

    def chunk_document(self, text: str, chunk_size: int = 512, overlap: int = 50) -> List[str]:
        """□□□□□□□□□□□□□□□□□"""
        words = text.split()
        chunks = []
        for i in range(0, len(words), chunk_size - overlap):
            chunk = " ".join(words[i:i + chunk_size])
            chunks.append(chunk)
        return chunks

    def build_from_directory(self, docs_dir: str):
        """□□□□□□□□□PDF/TXT/MD□□"""
        for filename in os.listdir(docs_dir):
            if filename.endswith(('.txt', '.md')):
                with open(os.path.join(docs_dir, filename), 'r', encoding='utf-8') as f:
                    text = f.read()
                chunks = self.chunk_document(text)
                embeddings = self.embedder.encode(chunks).tolist()

                # □□ChromaDB□□□□□□[reference:32][reference:33]
                self.collection.add(
                    documents=chunks,
                    embeddings=embeddings,
                    metadatas=[{
                        "source": filename,
                        "chunk_index": i,
                        "source_domain": "travel_knowledge"
                    } for i in range(len(chunks))],
                    ids=[f"{filename}_{i}" for i in range(len(chunks))]
                )
```

5.2 在线检索模块

python

```
# backend/app/rag/retriever.py

class RAGRetriever:
    def __init__(self, collection, embedder):
        self.collection = collection
        self.embedder = embedder

    def retrieve(self, query: str, top_k: int = 5) -> List[dict]:
        """□□□□□□□□□□□□□□□[reference:34]"""
        query_embedding = self.embedder.encode([query]).tolist()
        results = self.collection.query(
            query_embeddings=query_embedding,
            n_results=top_k,
            include=["documents", "metadatas", "distances"]
        )
        return [
            {
                "content": doc,
                "source": meta.get("source"),
                "score": 1 - dist  # □□□□□□
            }
            for doc, meta, dist in zip(
                results['documents'][0],
                results['metadatas'][0],
                results['distances'][0]
            )
        ]
```

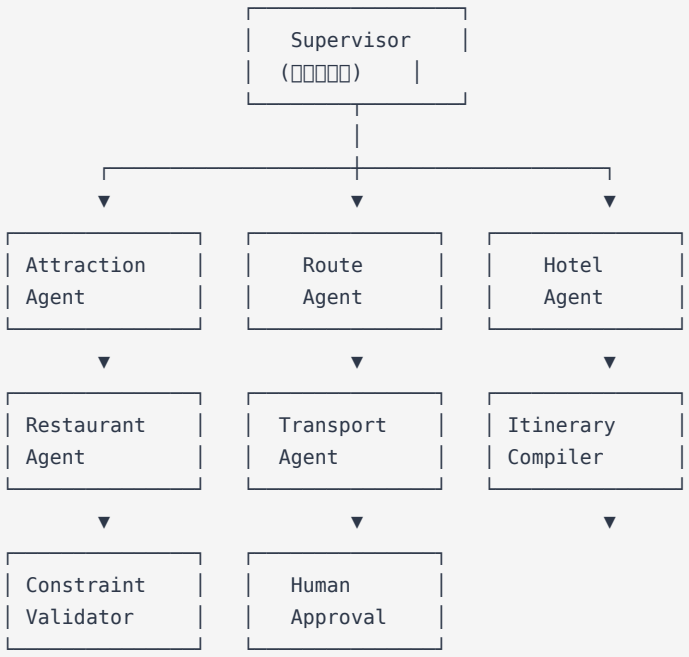
5.3 知识库数据来源建议

数据类型	来源	用途
景点介绍	WikiVoyage、百度百科	景点背景、开放时间
美食推荐	本地美食攻略、大众点评	餐厅推荐、特色菜
交通指南	官方旅游网站	机场交通、市内交通
文化贴士	领事馆网站、旅游博客	签证、习俗、安全
季节信息	气象局公开数据	最佳旅游季节、天气

六、多Agent协作实现 (LangGraph)

6.1 Supervisor模式架构

系统采用 动态Supervisor调度 模式：一个Supervisor Agent根据用户意图，动态路由到8个专职Agent：



6.2 LangGraph图构建

python

```python
# backend/app/graph/builder.py

from langgraph.graph import StateGraph, END
from langgraph.checkpoint import MemorySaver
from .state import AgentState
from .nodes import *
from .supervisor import supervisor_router

def build_travel_graph():
    # □□□StateGraph
    workflow = StateGraph(AgentState)

    # □□□□
    workflow.add_node("supervisor", supervisor_node)           # □□□□
    workflow.add_node("attraction", attraction_agent)          # □□□□
    workflow.add_node("route", route_agent)                   # □□□□
    workflow.add_node("hotel", hotel_agent)                   # □□□□
    workflow.add_node("restaurant", restaurant_agent)         # □□□□
    workflow.add_node("transport", transport_agent)           # □□□□
    workflow.add_node("itinerary_compiler", compiler_agent)   # □□□□
    workflow.add_node("validator", validator_agent)           # □□□□
    workflow.add_node("human_approval", human_agent)          # □□□□

    # □□□□
    workflow.set_entry_point("supervisor")

    # □□□□□Supervisor□□□□□□Agent[reference:42]
    workflow.add_conditional_edges(
        "supervisor",
        supervisor_router,
        {
            "attraction": "attraction",
            "route": "route",
            "hotel": "hotel",
            "restaurant": "restaurant",
```

```
            "transport": "transport",
            "compile": "itinerary_compiler",
            "end": END
        }
    )

    # □□Agent□□□□□□Supervisor
    for agent in ["attraction", "route", "hotel", "restaurant", "transport"]:
        workflow.add_edge(agent, "supervisor")

    # □□□→□□□→□□→□□
    workflow.add_edge("itinerary_compiler", "validator")
    workflow.add_edge("validator", "human_approval")
    workflow.add_edge("human_approval", END)

    # □□□□□□□□□□□□□□□□□□□□
    return workflow.compile(checkpointer=MemorySaver())
```

6.3 节点实现示例

python

```python
# backend/app/graph/nodes.py

from langchain_core.messages import HumanMessage, SystemMessage
from ..tools.weather import get_weather_forecast
from ..rag.retriever import RAGRetriever

def attraction_agent(state: AgentState) -> AgentState:
    """□□□□Agent□□□RAG□□□□□API"""
    request = state["request"]
    destination = request["destination"]

    # 1. RAG□□□□□□□□□□□□□
    rag = RAGRetriever()
    docs = rag.retrieve(f"{destination} □□□□ □□")

    # 2. □□LLM□□□□
    llm = get_llm()
    prompt = f"""
    □□□□□□□□{destination}□□5□□□□□□
    □□□□□□{docs}
    □□□□□{request.get('preferences', {})}
    □□□□□□□□□□□□□□□□□□□□□□□□□
    """
    response = llm.invoke(prompt)

    # 3. □□□□
    state["attractions"] = parse_attractions(response)
    state["messages"].append({"role": "assistant", "content": response})
    return state
```

七、外部API集成

7.1 实时数据API

数据类型	API	用途
航班	Amadeus Flight API	实时航班查询、比价
酒店	Amadeus Hotel API / Booking.com	酒店搜索、价格
景点	Google Places API	POI搜索、评分
天气	OpenWeatherMap	目的地天气预报
地图	高德地图 / Google Maps	路线规划、距离计算
货币	ExchangeRate API	实时汇率转换

7.2 工具函数示例

python

```
# backend/app/tools/weather.py

import aiohttp
from typing import Optional
from ..config import settings

async def get_weather_forecast(city: str, days: int = 7) -> dict:
    """□□□□□□□□□□"""
    url = f"https://api.openweathermap.org/data/2.5/forecast"
    params = {
        "q": city,
        "appid": settings.OPENWEATHER_API_KEY,
        "units": "metric",
        "cnt": days * 8  # □□8□3□□□□
    }
    async with aiohttp.ClientSession() as session:
        async with session.get(url, params=params) as resp:
            data = await resp.json()
            return {
                "city": data["city"]["name"],
                "forecast": [
                    {
                        "date": item["dt_txt"],
                        "temp": item["main"]["temp"],
                        "condition": item["weather"][0]["description"]
                    }
                    for item in data["list"][::8]  # □□□□□
                ]
            }
```

7.3 并行工具调用优化

python

```
# backend/app/graph/nodes.py - □□□□□□□□□

import asyncio

async def fetch_realtime_data(destination: str, dates: tuple):
    """□□asyncio.gather□□□□□□□□□□[reference:53]"""
```

```
    weather_task = get_weather_forecast(destination)
    flights_task = search_flights(destination, dates)
    hotels_task = search_hotels(destination, dates)

    # □□□□□□□□□□□□□□
    weather, flights, hotels = await asyncio.gather(
        weather_task, flights_task, hotels_task
    )
    return {"weather": weather, "flights": flights, "hotels": hotels}
```

八、流式响应 (SSE) 实现

8.1 后端SSE端点

python

```
# backend/app/api/stream.py

from fastapi import APIRouter, Request
from fastapi.responses import StreamingResponse
import json
import asyncio

router = APIRouter(prefix="/api/v1/trips", tags=["trips"])

@router.post("/stream")
async def stream_trip_plan(request: Request, trip_req: TravelPlanRequest):
    """SSE□□□□Agent□□□□[reference:56]"""

    async def event_generator():
        # □□□Agent
        graph = build_travel_graph()
        initial_state = {"request": trip_req.dict(), "messages": []}

        # □□□□
        async for event in graph.astream(initial_state):
            # □□□□□□SSE□□
            yield f"data: {json.dumps({'type': 'progress', 'data': event})}\n\n"

            # □□□□□□
            if "attractions" in event:
                yield f"data: {json.dumps({'type': 'attractions', 'data': event['attractions']})}\n\n"
            if "itinerary" in event:
                yield f"data: {json.dumps({'type': 'itinerary', 'data': event['itinerary']})}\n\n"

        # □□□□
        yield f"data: {json.dumps({'type': 'done'})}\n\n"

    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no"  # □□Nginx□□
        }
    )
```

8.2 前端SSE消费

typescript

```typescript
// frontend/src/api/trip.ts

export function streamTripPlan(request: TripRequest): EventSource {
    const eventSource = new EventSource(
        `${API_BASE}/api/v1/trips/stream`,
        { withCredentials: true }
    );

    eventSource.onmessage = (event) => {
        const data = JSON.parse(event.data);
        switch (data.type) {
            case 'progress':
                updateProgress(data.data);
                break;
            case 'attractions':
                displayAttractions(data.data);
                break;
            case 'itinerary':
                displayItinerary(data.data);
                break;
            case 'done':
                eventSource.close();
                break;
        }
    };

    eventSource.onerror = () => {
        eventSource.close();
    };

    return eventSource;
}
```

九、快速启动指南

9.1 环境准备

bash

```bash
# 1. □□□□
git clone https://github.com/YOUR_USERNAME/travel-planner-agent.git
cd travel-planner-agent

# 2. □□□□□□
cd backend
python -m venv venv
source venv/bin/activate  # Linux/Mac
# □ venv\Scripts\activate  # Windows

# 3. □□□□
pip install -r requirements.txt
```

9.2 配置环境变量

bash

```
# backend/.env
DEEPSEEK_API_KEY=sk-your-deepseek-key
GOOGLE_API_KEY=your-google-api-key      # □ Gemini
AMAP_API_KEY=your-amap-key              # □□□□
OPENWEATHER_API_KEY=your-weather-key
AMADEUS_API_KEY=your-amadeus-key        # □□□□□□□
AMADEUS_API_SECRET=your-amadeus-secret
SECRET_KEY=your-32-char-random-string   # JWT□□
DATABASE_URL=sqlite:///./travel.db      # □ postgresql://...
```

9.3 构建知识库

bash

```
# □□□□□□□□ data/raw/ □□
# □□□□□□□□□□□
python -m scripts.build_kb --input data/raw/ --output data/chromadb/
```

9.4 启动服务

bash

```
# □□
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# □□□□□□□□
cd ../frontend
npm install
npm run dev
```

9.5 Docker一键部署

bash

```
docker-compose up -d
# □□ http://localhost
```

十、API端点汇总

方法	路径	说明
POST	/api/v1/auth/register	用户注册
POST	/api/v1/auth/login	用户登录（返回JWT）
POST	/api/v1/trips/stream	SSE流式行程规划
GET	/api/v1/trips/{trip_id}	获取历史行程
PUT	/api/v1/trips/{trip_id}	更新行程（人工编辑）
POST	/api/v1/trips/{trip_id}/export	导出PDF行程单
GET	/api/v1/health	健康检查

十一、成本估算与优化建议

11.1 Token成本控制

环节	估算	优化策略
每次规划	5,000-15,000 tokens	使用DeepSeek等低成本模型
RAG检索	几乎免费（本地）	知识库本地化，无API调用费用
外部API	按调用量计费	缓存天气、汇率等静态数据

11.2 优化建议

RAG优先：能用知识库回答的，尽量不调LLM

结果缓存：相同目的地+日期的查询，缓存24小时

流式输出：SSE让用户感知更快，提升体验

免费额度：为用户设置每日免费查询次数，防止滥用